BCA Sem - 3
Programing using Java

Unit - 5
Database Connectivity



Introduction

JDBC stands for Java Database Connectivity. JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers to connect with the database.

Introduction

JDBC is a Java-based data access technology from Sun Microsystems.

It is an acronym as it is unofficially referred to as Java Database Connectivity, with DB being universally recognized as the abbreviation for database. This technology is an API for the Java programming language that defines how a client may access a database. It provides methods for querying and updating data in a database. JDBC is oriented towards relational databases. A JDBC-to-ODBC bridge enables connections to any ODBC-accessible data source in the JVM host environment.

Introduction

Sun Microsystems released JDBC as part of JDK 1.1 in the year of 1996. It has since formed part of the Java Standard Edition. The JDBC classes are contained in the Java package java.sql and javax.sql. JDBC has been updated several times as a part of the release of JDK 1.2 in 1998, a second version of JDBC was issued.

JDBC 3 is included with JDK 1.5 and 5.0.

Connection Interface

Connection interface defines connection to the different databases. An instance of the connection interface obtained from the getConnection() method of DriverManager class.

* Methods of Connection Interface

(1)createStatement(): It is used to create a statement object for sending the SQL statements to the database.

Syntax:

Statement st; St=con.createStatement();

(2)preparedStatement(): It is used to create a prepared sql statement object for sending sql statement to the database.

Syntax:

PreparedStatement pst; Pst=con.prepareStatement(sql);

Connection Interface

(3) prepareCall (Callable Statement) : It is used to create object for callable statement for calling stored procedure of database.
Syntax:
CallableStatement cst = con.prepareCall("{call insertdata (?,?)}");
Cst.setInt(1,1);
Cst.setString(2,'sname');

(4) Close(): It is used to release the connection. Syntax is con.close();

(5) commit(): It is used to save the states of the database which are changed by the transactions.
Syntax: con.commit();

(6) getMetaData(): It is used to get the metadata related to the current connection with the database. It provides the information like drivername, driver type, version and so on.
Syntax:
DatabaseMetaData ds = con.getMetaData();

Statement interface

This interface object is used to execute the SQL statements/queries and obtain the result procedure after executing the statement. It has a different types of methods like following

(a) executeQuery(): It is used to fetch the records from the database by using select query. It can only return the single ResultSet object at a time.

Syntax:

```
Statement st = con.createStatement()
ResultSet rs=st.executeQuery(“select * from tableName”);
```

Statement interface

(b) executeUpdate(): It is used to perform insert, update and delete operations on the records of the database. It returns the integer number which indicates the no.of records affected in the database after each transaction.

Syntax:

Statement st=con.createStatement();

Int n = st.executeUpdate("insert into tablename values(1,'abc','xyz')");

(c) execute(): It is used to execute the sql statements which returns the multiple results. It returns the Boolean indicating value.

Syntax:

Statement st = con.createStatement();

String sql = "create table tablename(fieldname type(size), fieldname type(size))";

St.execute(sql);

PreparedStatement interface

- The PreparedStatement interface is a subinterface of Statement.
- It is used to execute parameterized query. This interface is derived from the statement interface.
- Disadvantage of statement interface is that it executes the simple sql statements with no parameters and the sql statement is not precompiled.
- PreparedStatement object uses the template to create a SQL request and uses a prepared statement to send a precompiled sql statement with one or more parameters, so sometimes it is more convenient to use a preparedstatement object for sending sql statements to the database.
- If you want to execute a sql statement many times, preparedstatement object is very useful and it reduce the execution time compare to ordinary statement object.

ResultSet interface

- It is used to access the data from the table of the database.
- It stores the data of the table by executing the query.
- It also maintains the cursor position for the navigation operation within the large records. Cursor can move on the first, last, next and previous position from the current row position.
- It can also fetch the data either passing column name or index number with it’s relevant methods.
- In this case index number always starts from 1.
- There are few methods and static fields are used to perform the above operations as we have discussed.

ResultSet interface

Fields of ResultSet interface:

1. CONCUR_READ_ONLY: It defines that ResultSet object can not be modified or updated.

2. TYPE_FORWARD_ONLY: It defines that cursor from the current row can move forward only.this is default

3. TYPE_SCROLL_INSENSITIVE: it defines that cursor can scroll but can not be modified or updated.

4. TYPE_SCROLL_SENSITIVE: It defines that the cursor can scroll and also can be modified and updated.

MetaData

Meaning of metadata is data of data.
There are two types of metadata available in JDBC like
(1) ResultSet MetaData (2) Database MetaData

(1) ResultSet MetaData:
It has many methods which provides the info like number of columns, column name, sql type, size etc.

(2) Database MetaData:
It has many methods which provides the info like no.of tables, username, database mode, connection info, product name, version, driver name, driver version, max row size, max connection info etc.

Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

1. Register the Driver class
2. Create connection
3. Create statement
4. Execute queries
5. Close connection

1) Register the driver class

The forName() method of Class class is used to register the driver class.

This method is used to dynamically load the driver class.

Syntax of forName() method

public static void forName(String className)throws ClassNotFoundException

Example

```
Class.forName("oracle.jdbc.driver.OracleDriver");  //oracle
Class.forName("com.mysql.cj.jdbc.Driver"); //mysql
```

2) Create the connection object

The getConnection() method of DriverManager class is used to establish connection with the database.
Syntax of getConnection() method

public static Connection getConnection(String url,String name,String password) throws SQLException

example

```
Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe","system","password");  // Oracle
Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/business","root",""); //mysql
```

3) Create the Statement object

The createStatement() method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

Syntax of createStatement() method

public Statement createStatement()throws SQLException

example

Statement stmt=con.createStatement();

4) Execute the query

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

Syntax of executeQuery() method

public ResultSet executeQuery(String sql)throws SQLException

example

```
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next()){
System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

5) Close the connection object

By closing connection object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax of close() method

public void close()throws SQLException

Example
con.close();

PROGRAMS

INSERT

```
import java.sql.*;
class InsertRecord
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection
con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","")
;

		Statement stmt=con.createStatement();
		//String rec1="INSERT INTO EMP "+"VALUES(105,'AMIT',34000)";
		String rec1="insert into emp values(105,'AMIT',34000)";
		int rows=stmt.executeUpdate(rec1);
		System.out.println("Record Added..");
		con.close();
	}
```

```
catch(Exception e)
	{
		System.out.println(e);
	}

	}
}
```

UPDATE

```
import java.sql.*;
class UpdateRecord
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		Statement stmt=con.createStatement();
		//String rec1="UPDATE EMP set ename='AJAY',salary=20000"+" where empid=105";
		String rec1="update emp set ename='Jay',salary=39000 where empid=105";
		int rows=stmt.executeUpdate(rec1);
		System.out.println("Record Updated..");
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}

	}
}
```

DELETE

```
import java.sql.*;
class DeleteRecord
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		Statement stmt=con.createStatement();
		String rec1="delete from emp where empid=105";
		int rows=stmt.executeUpdate(rec1);
		System.out.println("Record Deleted..");
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}

	}
}
```

DISPLAY

```
import java.sql.*;
class DisplayRecord
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		Statement stmt=con.createStatement();
		ResultSet rs=stmt.executeQuery("select * from emp");
		while(rs.next())
			System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getInt(3));
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}
	}
}
```

CREATE TABLE

```
import java.sql.*;
class CreateTable
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		Statement stmt=con.createStatement();
		String rec1="create table student(sid integer(10),sname varchar(30))";
		stmt.execute(rec1);
		System.out.println("Table created..");
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}
	}
}
```

INSERT PREPARED

```
import java.sql.*;
class InsertPrepared
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		PreparedStatement stmt=con.prepareStatement("insert into emp values(?,?,?)");
		stmt.setInt(1,102);
		stmt.setString(2,"Hardik");
		stmt.setInt(3,55000);

		int rows=stmt.executeUpdate();
		System.out.println("Record Added..");
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}

	}
}
```

UPDATE PREPARED

```
import java.sql.*;
class UpdatePrepared
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		PreparedStatement stmt=con.prepareStatement("update emp set ename=?,salary=? where
empid=?");

		stmt.setString (1,"Jaypal");
		stmt.setInt(2,15000);
		stmt.setInt(3,102);

		int rows=stmt.executeUpdate();
		System.out.println("Record Updated..");
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}

	}
}
```

DELETE PREPARED

```
import java.sql.*;
class DeletePrepared
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		PreparedStatement stmt=con.prepareStatement("delete from emp where empid=?");
		stmt.setInt(1,105);

		int rows=stmt.executeUpdate();
		System.out.println("Record Deleted..");
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}

	}
}
```

DISPLAY PREPARED

```
import java.sql.*;
class DisplayPrepared
{
	public static void main(String args[]){
	try
	{
		Class.forName("com.mysql.cj.jdbc.Driver");
		Connection con=DriverManager.getConnection("jdbc:mysql://localhost:/employee","root","");

		PreparedStatement stmt=con.prepareStatement("select * from emp");
		ResultSet rs=stmt.executeQuery();
		while(rs.next())
			System.out.println(rs.getInt(1)+"  "+rs.getString(2)+"  "+rs.getInt(3));
		con.close();
	}
	catch(Exception e)
	{
		System.out.println(e);
	}
	}
}
```

Thank You